Claude Certified Architect - Foundations

Practice Question Bank · Set B · 60 scenario-based questions

This set contains 60 original, scenario-based multiple-choice questions, entirely different from Set A, covering all five domains of the Claude Certified Architect - Foundations exam. Scenarios include a customer support resolution agent, a multi-agent research system, Claude Code for teams and CI/CD, and structured data extraction. Question counts per domain follow the published exam weighting.

Domain	Exam weight	Questions in this set	Question numbers
D1. Agentic Architecture & Orchestration	27%	16	1-16
D2. Tool Design & MCP Integration	18%	11	17-27
D3. Claude Code Configuration & Workflows	20%	12	28-39
D4. Prompt Engineering & Structured Output	20%	12	40-51
D5. Context Management & Reliability	15%	9	52-60

How to use this set

- Treat it as a mock exam: 60 questions, 120 minutes, closed book, no AI help.
- Each question has one best answer. Several options are often partly true; choose the one that best addresses the root cause in the scenario.
- Mark your answers, then check the Answer Key and read every explanation, including the ones you got right.
- Record your score per domain on the score sheet at the end. Aim for at least 75% in every domain, not just overall.

Note: these are original practice questions written to match the publicly described exam blueprint (domains, weights, scenario style). They are not actual exam questions. Weights and format are as reported in October 2026; check Anthropic's official exam guide before you sit the exam.

Domain 1: Agentic Architecture & Orchestration

Exam weight 27% · 16 questions

QUESTION 1 · SCENARIO: CUSTOMER SUPPORT AGENT

Your agent loop ends as soon as an assistant message contains any text block. Users report that the agent says "Let me check your order..." and then stops without ever looking the order up. What is wrong?

- **A.** The model is too verbose. Add an instruction telling it never to write text before calling a tool.
- **B.** The tool is too slow. Make the order lookup asynchronous so that it finishes before the text is shown.
- **C.** The loop should end only when the response contains no text at all.
- **D.** Claude often returns text and a tool_use block in the same response. The loop should check `stop_reason` and keep going while it is `"tool_use"`.

QUESTION 2 · SCENARIO: CUSTOMER SUPPORT AGENT

To control costs, your team made `max_iterations = 5` the main way the agent loop ends. Complex cases such as multi-order disputes are now often cut off mid-resolution, leaving customers with half-finished actions. What is the best design?

- **A.** Lower the cap to 3 so that agents learn to solve problems more efficiently.
- **B.** Let `stop_reason == "end_turn"` decide normal completion, and keep a much higher iteration cap only as a safety net that fails gracefully (for example by escalating) when reached.
- **C.** Remove all caps and trust the model never to loop indefinitely.
- **D.** Keep the cap at 5 and have the agent apologise to the customer when it is reached.

QUESTION 3 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Your research system always runs all five subagents (search, academic search, news, data analysis, synthesis) for every query, including simple factual ones such as "When was the company founded?". Costs and latency are high. What is the best improvement?

- **A.** Cache every subagent's results permanently so repeated pipelines are cheaper.
- **B.** Remove the coordinator and send every query straight to the synthesis subagent.
- **C.** Have the coordinator assess each query's complexity and invoke only the subagents it needs, dynamically.
- **D.** Run all five subagents on a smaller, cheaper model.

QUESTION 4 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

You need a specialised `legal-review` subagent that can read documents but must never write files or run shell commands. How should you define it with the Claude Agent SDK?

- **A.** Give it every tool and add a system-prompt line saying it must not write files or run commands.
- **B.** Describe the restriction in the project CLAUDE.md so that all agents know legal review is read-only.
- **C.** Create an agent definition with a clear description, a role-specific system prompt and a tool list restricted to read-only tools.
- **D.** Use the same agent definition as the coordinator, but with a different name.

QUESTION 5 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

After a long conversation with a user, your coordinator passes its entire 150k-token history to each subagent it spawns. Subagents are slow and expensive, and they often go off on tangents from earlier in the conversation. What is the best fix?

A. Pass the full history but tell the subagent to focus only on the last message.

B. Pass each subagent only the context its subtask needs (goal, constraints, relevant prior findings) in a concise, structured form.

C. Pass nothing, and let each subagent rediscover the context with its own tools.

D. Summarise the history into a single sentence for every subagent.

QUESTION 6 · SCENARIO: CUSTOMER SUPPORT AGENT

Compliance rules say the agent must never send email to addresses outside the company and customer domains on file. The rule is in the system prompt, but a red-team test got the agent to email an attacker's address. What is the most robust control?

A. Add several more red-team examples to the system prompt to show the model what attacks look like.

B. Ask the model to confirm in its reasoning that each recipient is allowed before it sends.

C. A pre-tool-use hook that checks the recipients of every `send_email` call against the allowed domains, blocks any violation and explains why to the model.

D. Log all outgoing emails so that compliance can review them the next day.

QUESTION 7 · SCENARIO: CUSTOMER SUPPORT AGENT

When your agent escalates to a human, agents in the human queue can't see the AI conversation transcript. They have to ask customers to repeat everything, and satisfaction scores fall. What should the escalation step produce?

A. A note telling the human agent to ask the customer to describe the issue again.

B. A structured handoff summary with the customer ID, the issue and likely root cause, key amounts and dates, what was tried and the outcomes, and a recommended next action.

C. The raw, unedited JSON of every tool call made during the conversation.

D. A one-line label such as "Refund issue - escalated".

QUESTION 8 · SCENARIO: CUSTOMER SUPPORT AGENT

Incoming tickets fall into four well-understood categories, each handled by a different specialised prompt and toolset. Category boundaries are clear and stable. Which pattern fits best?

A. One general agent with all four toolsets, deciding everything for itself on every ticket.

B. A routing workflow: classify the ticket first, then send it to the specialised handler for that category.

C. Four agents in parallel on every ticket, keeping whichever answer looks best.

D. A single prompt that lists all four procedures and asks Claude to follow the right one.

QUESTION 9 · SCENARIO: DEVELOPER PRODUCTIVITY

A user asks your coding agent to "rename the `Account` model to `Customer` everywhere". How many files are affected, and which ones, can only be found by searching the codebase. Which pattern is most appropriate?

A. A fixed prompt chain with exactly five steps defined in advance.

B. Parallelisation across a hard-coded list of directories chosen before the search runs.

C. Orchestrator-workers: the orchestrator finds the affected files at runtime and hands per-file work to workers, then checks the combined result.

D. A single call asking Claude to output the whole updated codebase.

QUESTION 10 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Your subagents return long narrative prose. The coordinator struggles to merge them, and claims lose their source attribution along the way. What is the best change?

A. Ask subagents to write shorter, punchier narratives.

B. Have the coordinator ask each subagent follow-up questions to recover any missing sources.

C. Have the synthesis subagent read all source documents again from scratch.

D. Require subagents to return structured output, such as a list of claims, each with its evidence excerpt, source URL and publication date.

QUESTION 11 · SCENARIO: DEVELOPER PRODUCTIVITY

Yesterday you ran a named Claude Code investigation into an authentication bug. Since then, a teammate has changed two of the twenty files you analysed. What is the most efficient way to continue?

A. Start a completely fresh session and repeat the whole investigation.

B. Resume the session without mentioning the changes, since Claude will notice them automatically.

C. Fork the session and run the old and new analyses side by side.

D. Resume the named session and tell Claude exactly which two files changed so that it re-analyses them specifically.

QUESTION 12 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Four of five parallel research subagents succeed. The fifth (patent search) fails even after retrying locally. What should the coordinator do next?

A. Discard all results and restart the whole workflow from the beginning.

B. Have the synthesis subagent make plausible estimates of the patent findings so the report is complete.

C. Go ahead with synthesis using the four successful results, optionally try an alternative approach for the patent search, and state the patent coverage gap clearly in the report.

D. Hold the report until the patent source is back online, however long that takes.

QUESTION 13 · SCENARIO: CUSTOMER SUPPORT AGENT

Your support agent's tool results sometimes include customers' full payment card numbers and national ID numbers from a legacy system. You must make sure these never enter the model's context or logs. What is the best approach?

A. A post-tool-use hook (or tool-side filtering) that redacts the sensitive fields from results before they are added to the conversation.

B. A system-prompt instruction telling Claude never to repeat card numbers in its replies.

C. Ask customers not to have card numbers stored in their accounts.

D. Encrypt the conversation logs at rest so the data is protected after it is stored.

QUESTION 14 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Traces show three research subagents fetching and analysing many of the same sources, which duplicates cost and repeats content in the report. What is the root-cause fix?

A. Have the coordinator give each subagent an explicit, non-overlapping scope (for example by subtopic or source type) in its prompt.

B. De-duplicate the final report at synthesis time and leave delegation as it is.

C. Run the subagents one after another so each one can see what the earlier ones found.

D. Cut the number of subagents to one so overlap is impossible.

QUESTION 15 · SCENARIO: CUSTOMER SUPPORT AGENT

Your team is choosing between a hard-coded decision tree and an agent that picks its own tools for handling returns. Return cases vary a lot, but two invariants must always hold: identity verification before any account action, and refunds capped at the original payment amount. What design is best?

A. A hard-coded decision tree covering every return scenario the team can think of.

B. A fully autonomous agent, with both invariants described clearly in the system prompt.

C. A decision tree for verification and a free-form agent for everything else, with no checks on refunds.

D. Let the model choose tools based on the context of each case, and enforce the two invariants programmatically (for example with prerequisite gates and argument checks).

QUESTION 16 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

A search subagent keeps calling `web_search` with the same query, getting the same results, and trying again. Within one task it has made 14 identical calls. What is the best mitigation?

A. Raise the subagent's temperature so that it naturally picks different queries.

B. Silently return random results for repeated queries so the subagent sees something new.

C. Detect repeated identical tool calls and return a tool result saying no new results are available and suggesting a different query or approach, with an overall iteration cap as a backstop.

D. Remove the web search tool from the subagent after its first call.

Domain 2: Tool Design & MCP Integration

Exam weight 18% · 11 questions

QUESTION 17 · SCENARIO: CUSTOMER SUPPORT AGENT

Your `lookup_order` tool description is detailed and accurate, but the agent keeps calling `search_knowledge_base` for questions about order status. The system prompt says: "Always check the knowledge base first before answering any question." What is the most likely cause?

A. The system prompt's blanket instruction overrides the tool descriptions. Remove or narrow it so that it doesn't conflict with correct tool selection.

B. The `lookup_order` description is too long, and Claude ignores tools with long descriptions.

C. Claude always prefers tools whose names start with "search".

D. The knowledge-base tool must come after `lookup_order` in the tools array.

QUESTION 18 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Your system has `search_docs` ("Searches documents") and `search_wiki` ("Searches wiki content"). The wiki is also indexed in the document store, so both tools often return overlapping results, and the agent picks between them inconsistently. What is the best fix?

A. Have the agent always call both tools and merge the results.

B. Add a random tie-breaker so selection evens out between the two tools.

C. Redefine and rename the tools so their purposes don't overlap (for example, scope one to the product-specification store and one to the internal how-to wiki), and say in each description when to use it instead of the other.

D. Keep both tools as they are and add a third tool that searches both sources.

QUESTION 19 · SCENARIO: CUSTOMER SUPPORT AGENT

A customer asks for a $900 refund, and the `process_refund` tool rejects it because policy caps automatic refunds at $500. How should the tool report this, and what should the agent do?

A. Return a transient error so that the agent retries the refund a few times.

B. Return a success response for $500 and silently drop the rest of the request.

C. Return a generic "Request failed" so as not to reveal internal policy to the agent.

D. Return a structured business-rule error that is marked as not retryable and includes a customer-friendly reason. The agent explains the limit and offers escalation or other options.

QUESTION 20 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

A search subagent's API call returns HTTP 503 (Service Unavailable). Where should this error be handled first?

A. Inside the subagent: retry locally with backoff, and pass the error up to the coordinator, with context and any partial results, only if it can't be recovered.

B. Pass it straight to the coordinator, which should decide on every retry for every subagent.

C. Pass it all the way to the end user and ask them to try again later.

D. Ignore it, and return whatever results were cached last week.

QUESTION 21 · SCENARIO: CLAUDE CODE FOR TEAMS

Your team wants Claude Code to work with Jira. There's a well-maintained community MCP server for Jira, but one engineer wants to build a custom one. When is building a custom MCP server more justified?

A. Always. Custom servers are required for any production integration.

B. Whenever a community server exists, since community servers are generally less secure than custom ones.

C. Only when the community server has fewer tools than the Jira REST API has endpoints.

D. When you need team-specific workflows or internal systems that existing servers don't cover. Standard integrations like Jira should normally use the existing server.

QUESTION 22 · SCENARIO: CLAUDE CODE FOR TEAMS

A developer has two MCP servers configured in the project's `.mcp.json` and one in their personal user-level configuration. Which tools are available to Claude Code in that project?

A. Only the project servers. Project scope always replaces user scope.

B. Only the user server, because personal settings take priority over project settings.

C. Tools from all three servers, discovered when Claude Code connects and available at the same time.

D. Only one server at a time, chosen by running a command at the start of the session.

QUESTION 23 · SCENARIO: DEVELOPER PRODUCTIVITY

Your team added an MCP code-search tool that understands symbols and cross-references, but Claude Code keeps using the built-in Grep for code questions instead. What is the most effective fix?

A. Uninstall Grep from Claude Code so that only the MCP tool is left.

B. Rewrite the MCP tool's description to explain what it can do (symbol-aware search, cross-references, what it returns) and when to prefer it over plain text search.

C. Rename the MCP tool to `grep` so that Claude picks it by name.

D. Tell developers to type the MCP tool's name in every prompt.

QUESTION 24 · SCENARIO: DEVELOPER PRODUCTIVITY

You need a list of every Terraform file in a large monorepo so that you can review module versions. Which built-in Claude Code tool should you use first?

A. Grep for the word "terraform" across every file.

B. Read the repository's root directory and follow every subdirectory by hand.

C. Bash, running `cat` on every file in the repository.

D. Glob with the pattern `**/*.tf`.

QUESTION 25 · SCENARIO: DEVELOPER PRODUCTIVITY

Claude is new to a large codebase and needs to understand how payments flow through it. Which exploration strategy is most effective?

A. Read every file in the repository first so that Claude has complete context before reasoning.

B. Ask Claude to guess the architecture from the repository name and README alone.

C. Grep for entry points (such as the payment route handlers), then Read along the imports and calls step by step, building understanding as you go.

D. Use Glob to list every file, then Read them in alphabetical order.

QUESTION 26 · SCENARIO: CUSTOMER SUPPORT AGENT

To "guarantee structured output", a developer set `tool_choice` to `any` on every turn of the conversational support agent. Now the agent can't ask the customer clarifying questions or give a plain answer. Which setting fits this agent?

A. `any`, with an extra `ask_question` tool used only for clarifying questions.

B. `auto`, which lets Claude decide whether to call a tool or reply with text.

C. A forced specific tool on every turn, rotated by turn number.

D. `none` on every turn, with tool calls detected by parsing the text.

QUESTION 27 · SCENARIO: CUSTOMER SUPPORT AGENT

A customer says "ship it next Tuesday", and the agent passes `"next Tuesday"` to `schedule_delivery`, whose `date` parameter expects `YYYY-MM-DD`. How should the tool respond?

A. Return a validation error that states the expected format, with an example, so the agent can fix the input and retry.

B. Return a generic 500 error so that the agent escalates to a human.

C. Quietly schedule the delivery for today's date as the closest valid default.

D. Return success and log the bad input for engineers to look at later.

Domain 3: Claude Code Configuration & Workflows

Exam weight 20% · 12 questions

QUESTION 28 · SCENARIO: CLAUDE CODE FOR TEAMS

Your team is choosing between path-scoped rule files in `.claude/rules/` and per-directory CLAUDE.md files for conventions about database migration files, which live in `services/*/migrations/` across 12 services. What is the main advantage of path-scoped rules here?

A. One rule with a glob such as `services/*/migrations/**` applies to every matching file wherever it sits, so you don't need 12 duplicated directory-level files.

B. Path-scoped rules are always loaded in every session, so Claude never misses them.

C. Path-scoped rules override every other memory file, including the user's personal CLAUDE.md.

D. Path-scoped rules are the only way to give Claude conventions for files outside the repository root.

QUESTION 29 · SCENARIO: CLAUDE CODE FOR TEAMS

A developer likes Claude to explain its reasoning in detail before every change. Teammates don't want that. Where should this preference go?

A. In the developer's user-level `~/.claude/CLAUDE.md`.

B. In the project's root `CLAUDE.md`, under a heading with the developer's name.

C. In a rule file under `.claude/rules/` scoped to `**/*`.

D. In the project's committed `.claude/settings.json`.

QUESTION 30 · SCENARIO: CLAUDE CODE FOR TEAMS

Your 300-line release checklist sits in the root CLAUDE.md, so it loads in every session even though it's used about once a week. Developers notice it takes up context. What is the best change?

A. Move the checklist into a skill (for example `.claude/skills/release/SKILL.md`) that is invoked when needed.

B. Cut the checklist down to 20 lines so it costs less context.

C. Move it to each developer's user-level CLAUDE.md.

D. Leave it in place and run `/compact` at the start of every session.

QUESTION 31 · SCENARIO: CLAUDE CODE FOR TEAMS

You're writing a `codebase-report` skill that should only ever read and search files. It must never modify the repository, even by accident. What should you add?

A. An `allowed-tools` entry in the SKILL.md frontmatter listing only read and search tools.

B. A sentence in the skill body saying "Do not modify any files".

C. A note in CLAUDE.md saying the report skill is read-only.

D. A git pre-commit hook that rejects commits made while the skill is running.

QUESTION 32 · SCENARIO: CLAUDE CODE FOR TEAMS

Your `/triage` skill needs a ticket ID to work. Developers often run it with no argument, and Claude then guesses or asks vague questions. Which frontmatter field helps most?

A. `context: fork`, so that the skill runs in an isolated context.

B. `allowed-tools`, limited to the ticket-system tools.

C. `model`, set to the most capable model so that it can infer the missing ticket.

D. `argument-hint`, which tells people the skill expects a ticket ID (for example `<ticket-id>`) when they invoke it.

QUESTION 33 · SCENARIO: CLAUDE CODE FOR TEAMS

Your team shares a `/review` skill in the repository. You want a personal variant with extra checks for your own experiments, without changing what teammates get. What should you do?

A. Edit the shared skill in the repository and push your extra checks.

B. Create a personal skill named `review` so it silently replaces the team's version on your machine.

C. Create a personal skill under `~/.claude/skills/` with a different name, such as `review-strict`.

D. Add your extra checks to the project CLAUDE.md under a section labelled "personal".

QUESTION 34 · SCENARIO: DEVELOPER PRODUCTIVITY

While planning a major refactor, Claude's codebase exploration produces a lot of verbose output, and the main session's context fills up before planning is finished. What is the most effective approach?

A. Turn plan mode off and start making changes, which uses less context than exploring.

B. Paste the whole codebase into the prompt at the start so no exploration is needed.

C. Delegate the exploration to the Explore subagent so the verbose discovery stays in its own context and only a summary comes back.

D. Keep exploring in the main session and accept that the session will reset when it fills up.

QUESTION 35 · SCENARIO: DEVELOPER PRODUCTIVITY

You need to add cache invalidation to a service, but you're not familiar with the caching layer's failure modes. How can you best use Claude Code before any code is written?

A. Ask Claude to implement the standard approach straight away and fix problems in code review.

B. Write a detailed specification yourself first, without Claude's help.

C. Ask Claude for three implementations and merge whichever parts look best.

D. Ask Claude to interview you: have it ask questions that bring out considerations such as consistency requirements, TTLs and failure behaviour before it proposes a design.

QUESTION 36 · SCENARIO: DEVELOPER PRODUCTIVITY

You want Claude to implement a tricky date-parsing function and be confident it handles every case. Which workflow is most effective?

A. Write tests first that cover the expected behaviour and edge cases, then have Claude implement the function and iterate by feeding it the test failures.

B. Have Claude implement it, then ask Claude whether it believes the implementation is correct.

C. Have Claude implement it and run it once on a single typical date.

D. Write the tests after implementation, matching whatever the code does.

QUESTION 37 · SCENARIO: DEVELOPER PRODUCTIVITY

You've found three bugs in a checkout module: a rounding error, a discount applied twice, and tax being calculated before the discount. Fixing any one of them changes how the others behave. How should you hand them to Claude?

A. One at a time in separate sessions, so that each fix is isolated.

B. Only the most severe bug, waiting to see whether the others go away.

C. In a single detailed message that describes all three and how they interact, so that the fixes are designed together.

D. All three as separate parallel subagent tasks with no shared context.

QUESTION 38 · SCENARIO: CLAUDE CODE IN CI/CD

Your CI job uses Claude Code to generate unit tests for changed code. It often creates tests that duplicate existing ones and ignores your shared fixtures. What is the best fix?

A. Include the relevant existing test files in context, and document your testing standards and available fixtures in the project CLAUDE.md, which the CI run reads.

B. Run the generated tests twice and delete any that pass both times.

C. Raise the job's timeout so that Claude has longer to find existing tests on its own.

D. Have a developer delete duplicate tests by hand after every CI run.

QUESTION 39 · SCENARIO: CLAUDE CODE FOR TEAMS

Your team wants the formatter to run automatically after every file Claude Code edits, with no exceptions, and without relying on Claude remembering to do it. What is the best mechanism?

A. A post-tool-use hook in the project's `.claude/settings.json`, matching the Edit and Write tools, that runs the formatter.

B. An instruction in CLAUDE.md saying "Always run the formatter after editing files".

C. A skill named `/format` that developers run before they commit.

D. A slash command that tells Claude to format every file in the repository.

Domain 4: Prompt Engineering & Structured Output

Exam weight 20% · 12 questions

QUESTION 40 · SCENARIO: CLAUDE CODE IN CI/CD

Your review prompt includes "Check that comments are accurate." The reviewer now flags hundreds of harmless comments for minor wording problems. What is the best rewrite?

A. "Check very carefully that comments are accurate, and only report important issues."

B. "Flag a comment only when what it says the code does contradicts what the code actually does." Add an example of a contradiction and an example of acceptable imprecision.

C. "Report every comment that could be improved, but mark all of them low severity."

D. Remove the comment check entirely, since comments don't affect runtime behaviour.

QUESTION 41 · SCENARIO: CLAUDE CODE IN CI/CD

Reviewers notice that similar issues get different severity labels from one PR to the next. A null dereference in error handling is "critical" in one PR and "minor" in another. What should you change?

A. Ask Claude to think harder about severity before it assigns a label.

B. Define explicit criteria for each severity level, with concrete code examples for each.

C. Remove severity labels and present every finding as equally important.

D. Average the severity across three runs of the same review.

QUESTION 42 · SCENARIO: CUSTOMER SUPPORT AGENT

Your agent often has to choose between `issue_store_credit` and `escalate_to_billing` in borderline cases, and its choices are inconsistent. What prompting technique helps most?

A. A rule telling the agent to choose store credit whenever it is unsure.

B. A longer description of each tool, with no examples.

C. Few-shot examples of borderline cases that show the reasoning for choosing one action over a plausible alternative.

D. A higher temperature, so that the agent considers more options.

QUESTION 43 · SCENARIO: STRUCTURED DATA EXTRACTION

You extract quantities from recipes. Some say "2 cups", some "a handful", some "about 200g (roughly 1.5 cups)". The model often leaves out informal quantities or turns them into confident made-up numbers. What is the most effective fix?

A. Add a rule that informal quantities must always be converted to grams.

B. Drop recipes that contain any informal quantity.

C. Make the quantity field required so the model can't leave it out.

D. Add few-shot examples that show how to extract each kind of quantity, including recording informal amounts as stated, with a flag, rather than converting them.

QUESTION 44 · SCENARIO: STRUCTURED DATA EXTRACTION

You added three few-shot examples to your extraction prompt. All three are invoices from the same vendor template. Accuracy improved on that vendor but fell on others. What is the best fix?

A. Add seven more examples from the same vendor to strengthen the pattern.

B. Remove all the examples and rely on instructions alone.

C. Train a separate classifier to detect that vendor and use the examples only for it.

D. Replace them with a diverse set of examples covering different layouts and edge cases, so the model learns the general task rather than one template.

QUESTION 45 · SCENARIO: STRUCTURED DATA EXTRACTION

Your extraction output passes schema validation, but dates appear as `2026-03-04`, `March 4, 2026` and `04/03/26`, and currencies as `$`, `USD` and `US dollars`. What is the most direct fix?

A. Add explicit normalisation rules to the prompt and schema (for example ISO 8601 dates and ISO 4217 currency codes, with format patterns or enums where possible), with examples.

B. Accept the variation and leave normalisation to each team that uses the data.

C. Add more fields to the schema so that each date format has its own field.

D. Lower the temperature so the model picks a single format by chance.

QUESTION 46 · SCENARIO: STRUCTURED DATA EXTRACTION

Semantic validation finds an error in an extraction. To give the retry the best chance of success, what should the retry request include?

A. Only the original document, with the same prompt as before.

B. Only the validation errors, without the original document, to save tokens.

C. A message saying "That was wrong, try again", added to the conversation.

D. The original document, the extraction that failed, and the specific validation errors found.

QUESTION 47 · SCENARIO: STRUCTURED DATA EXTRACTION

Some invoices state a total that doesn't match the sum of their line items, either because the vendor made a mistake or because the extraction did. You want the pipeline to tell these apart and flag them. What schema design helps?

A. Extract only `total`, telling the model to fix any arithmetic errors it notices.

B. Extract both `stated_total` and `calculated_total` (the sum of the line items), plus a `totals_conflict` flag, so that mismatches are visible and can be routed for review.

C. Extract only line items and always compute the total downstream.

D. Drop any invoice whose totals don't match.

QUESTION 48 · SCENARIO: CUSTOMER SUPPORT AGENT

You want to cut costs on an agent pipeline in which each request involves several rounds of tool calls whose results depend on live systems. Someone suggests moving it to the Message Batches API. What is the main obstacle?

A. Requests in the Batches API can't include tool definitions at all.

B. The Batches API costs more than synchronous calls for agent workloads.

C. The Batches API always returns results in under a minute, which is too fast for live systems.

D. A batch request can't run a multi-turn tool loop: you can't run a tool and return its result within the same batch request, so agent loops don't fit the batch model.

QUESTION 49 · SCENARIO: STRUCTURED DATA EXTRACTION

Every night you classify about 10,000 support transcripts for weekly trend reports. Nobody needs the results before the next morning. Which approach best balances cost and requirements?

A. Run 10,000 synchronous requests in parallel at the highest possible rate.

B. Classify transcripts one at a time synchronously over the course of the night.

C. Skip classification and sample 100 transcripts by hand each week.

D. Submit them through the Message Batches API for the roughly 50% cost saving, since the workload tolerates delay.

QUESTION 50 · SCENARIO: CLAUDE CODE IN CI/CD

Your multi-pass reviewer outputs a confidence value with each finding. You want to send low-confidence findings to human reviewers. What must you do before you can trust the routing threshold?

A. Nothing. The model's confidence values are probabilities and can be used as they are.

B. Set the threshold at 0.5, since that is the midpoint between right and wrong.

C. Ask the model to be more honest about its confidence in the prompt.

D. Calibrate the confidence values against a labelled validation set, so that you know how the scores actually relate to correctness.

QUESTION 51 · SCENARIO: STRUCTURED DATA EXTRACTION

Your prompt contains a 60-page contract, extraction instructions and a question. Results get worse when the instructions are at the top and the contract is in the middle. Which structure works best for long documents?

A. Put the long document near the top, wrapped in clear XML-style tags, and put the instructions and question after it, at the end of the prompt.

B. Repeat the instructions after every page of the contract.

C. Split the contract into 60 separate requests and extract from each page without any cross-page context.

D. Put the instructions at the very top and the question in the middle of the document.

Domain 5: Context Management & Reliability

Exam weight 15% · 9 questions

QUESTION 52 · SCENARIO: DEVELOPER PRODUCTIVITY

Two hours into a codebase exploration session, Claude starts referring to "the typical pattern for services like this" instead of the specific classes it found earlier. What is the best way to keep the session reliable?

A. Record key findings in a scratchpad file as you go and refer to it, give large sub-investigations to subagents, and use `/compact` deliberately.

B. Keep going in the same session, since the model will recall the details if asked directly.

C. Restart the session every 15 minutes regardless of progress.

D. Paste the earlier transcript back into the conversation to refresh Claude's memory.

QUESTION 53 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

A long-running multi-agent research job crashes after three hours, and restarting from scratch is expensive. Which design would have let it recover?

A. Rely on the model to remember earlier progress when the job restarts.

B. Have each agent save its state (completed subtasks, findings, pending work) to a structured manifest. On restart, the coordinator loads the manifest and passes the relevant state into the agents' prompts.

C. Make the job run faster so that crashes are less likely.

D. Log the raw output of every API call and replay all of it on restart.

QUESTION 54 · SCENARIO: CUSTOMER SUPPORT AGENT

Audits show your agent escalates many easy password-reset cases while confidently handling complex billing disputes it gets wrong. Escalation currently happens when the model's self-rated confidence is below 0.6. What is the best fix?

A. Raise the confidence threshold to 0.8 so that more cases escalate.

B. Replace the confidence threshold with explicit escalation criteria based on case characteristics, and add few-shot examples of when to escalate and when to resolve.

C. Ask the model to rate its confidence twice and use the lower value.

D. Escalate every billing case and resolve every password case automatically.

QUESTION 55 · SCENARIO: CUSTOMER SUPPORT AGENT

A customer writes angrily: "This is the THIRD time my package is late!!" Their issue is a delayed shipment, which the agent can fix by reshipping with express delivery. They have not asked for a human. What should the agent do?

A. Escalate immediately, because the customer's sentiment is strongly negative.

B. Ignore the emotional tone and answer only the shipping question.

C. Acknowledge the frustration and offer the concrete fix straight away, and escalate if the customer then says they would prefer a human.

D. Ask the customer to calm down before going on with the request.

QUESTION 56 · SCENARIO: CUSTOMER SUPPORT AGENT

A customer asks the agent to match a competitor's lower price. Company policy covers refunds, returns and warranty, but says nothing about price matching. What should the agent do?

A. Agree to the price match, since the policy doesn't forbid it.

B. Refuse the price match, since the policy doesn't allow it.

C. Escalate to a human, because this is a gap in policy that needs a human decision. Tell the customer it is being reviewed.

D. Offer a partial refund as a compromise that the agent works out itself.

QUESTION 57 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Your research report states every finding with equal certainty, but some come from several strong sources and others from one blog post. Some subtopics couldn't be searched at all because a source was down. How should synthesis present this?

A. Present everything with equal certainty so the report reads confidently and consistently.

B. Mark which findings are well supported and which rest on weak or single sources, and state which areas weren't covered because sources were unavailable.

C. Leave out every finding that has only one source and say nothing about coverage.

D. Add a general disclaimer at the end saying that some findings may be inaccurate.

QUESTION 58 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Two subagents report adoption rates of 34% and 51% for the same technology. Investigation shows one figure is from 2022 and the other from 2025. What design change would have made this easy to interpret?

A. Have synthesis always report the higher figure, since adoption usually grows.

B. Require subagents to include publication or data-collection dates with every claim in their structured output, so that synthesis can see the figures differ because of time, not because one is wrong.

C. Have subagents drop any statistic older than one year without saying so.

D. Have synthesis average all reported figures for the same metric.

QUESTION 59 · SCENARIO: STRUCTURED DATA EXTRACTION

Your extraction pipeline gives each field a confidence score. Human reviewers can only check about 10% of documents. How should you decide which documents they see?

A. Send a random 10% to review regardless of confidence.

B. Route documents with low-confidence or ambiguous fields (using thresholds calibrated on labelled validation data) to review, plus a random sample of high-confidence ones to catch blind spots.

C. Review only the documents the model rated lowest overall, with no sampling of the others.

D. Review only the documents from the newest customers.

QUESTION 60 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Your synthesis agent's context fills up because upstream subagents send back long reasoning traces along with their findings. Quality drops in the final reports. What is the best fix?

A. Move the synthesis agent to a larger context window and keep sending the full traces.

B. Have upstream agents return compact, structured results (key facts, sources and any flags) and leave out their full reasoning, so the downstream context stays focused.

C. Have synthesis read only the first half of each subagent's output.

D. Remove the synthesis step and concatenate the subagents' outputs as the final report.

Answer Key

Quick reference. Detailed explanations follow.

1	D	**11**	D	**21**	D	**31**	A	**41**	B	**51**	A
2	B	**12**	C	**22**	C	**32**	D	**42**	C	**52**	A
3	C	**13**	A	**23**	B	**33**	C	**43**	D	**53**	B
4	C	**14**	A	**24**	D	**34**	C	**44**	D	**54**	B
5	B	**15**	D	**25**	C	**35**	D	**45**	A	**55**	C
6	C	**16**	C	**26**	B	**36**	A	**46**	D	**56**	C
7	B	**17**	A	**27**	A	**37**	C	**47**	B	**57**	B
8	B	**18**	C	**28**	A	**38**	A	**48**	D	**58**	B
9	C	**19**	D	**29**	A	**39**	A	**49**	D	**59**	B
10	D	**20**	A	**30**	A	**40**	B	**50**	D	**60**	B

Answers and Explanations

Why the best answer is best, and why the other options fall short.

Domain 1: Agentic Architecture & Orchestration

Question 1 · Answer: D

Claude often returns text and a tool_use block in the same response. The loop should check `stop_reason` *and keep going while it is* `"tool_use"`*.*

A single response can contain explanatory text and a tool call. `stop_reason` is the signal for whether Claude is waiting on a tool result. Stopping because text appeared drops the pending call. Banning preamble text treats the symptom. Speed is not the problem. A response with no text at all isn't a valid end condition either.

Question 2 · Answer: B

Let `stop_reason == "end_turn"` *decide normal completion, and keep a much higher iteration cap only as a safety net that fails gracefully (for example by escalating) when reached.*

An iteration cap is a guardrail, not the normal way a loop should end. The model's own end signal should end normal runs, and the cap should catch pathological loops, with a graceful fallback. Lower caps make truncation worse. Having no cap at all risks runaway cost. Apologising doesn't fix half-done actions.

Question 3 · Answer: C

Have the coordinator assess each query's complexity and invoke only the subagents it needs, dynamically.

A coordinator's job includes routing: a simple question doesn't need the whole pipeline. Permanent caching gives stale answers and doesn't help new queries. Removing the coordinator loses orchestration for complex queries. A cheaper model still runs work that isn't needed.

Question 4 · Answer: C

Create an agent definition with a clear description, a role-specific system prompt and a tool list restricted to read-only tools.

Agent definitions let you set each subagent's description (used for delegation), prompt and allowed tools. Restricting tools enforces read-only behaviour structurally. Prompt-only restrictions can be ignored. CLAUDE.md is shared guidance, not per-agent tool permissions. Copying the coordinator's definition would give it the coordinator's tools.

Question 5 · Answer: B

Pass each subagent only the context its subtask needs (goal, constraints, relevant prior findings) in a concise, structured form.

Subagents exist partly to keep each context focused. Curated, task-specific context reduces cost and keeps the subagent on topic. Sending the full history with a "focus" instruction still pays for and is distracted by irrelevant context. Sending nothing wastes effort rediscovering things. A one-sentence summary usually leaves out crucial constraints.

Question 6 · Answer: C

A pre-tool-use hook that checks the recipients of every `send_email` *call against the allowed domains, blocks any violation and explains why to the model.*

Security and compliance rules that must never be broken need deterministic enforcement outside the model. A hook that inspects tool arguments can't be talked round by prompt injection. Examples and self-checks lower the risk but can still fail. Reviewing logs the next day only detects the problem after the damage is done.

Question 7 · Answer: B

A structured handoff summary with the customer ID, the issue and likely root cause, key amounts and dates, what was tried and the outcomes, and a recommended next action.

A good handoff is self-contained and actionable for a reader who has no other context. Making the customer repeat themselves is the problem being solved. Raw tool JSON is too noisy to act on quickly. A one-line label leaves out what the human needs.

Question 8 · Answer: B

A routing workflow: classify the ticket first, then send it to the specialised handler for that category.

When inputs fall into clear, stable categories, routing lets each handler be optimised separately and keeps tool sets small. A general agent with every tool hurts selection. Running all four in parallel wastes resources. One giant prompt dilutes each specialised instruction.

Question 9 · Answer: C

Orchestrator-workers: the orchestrator finds the affected files at runtime and hands per-file work to workers, then checks the combined result.

Orchestrator-workers suits tasks where the subtasks can't be predicted until runtime. A fixed chain or a hard-coded list assumes you already know the scope. Regenerating the whole codebase in one output isn't feasible and can't be reviewed.

Question 10 · Answer: D

Require subagents to return structured output, such as a list of claims, each with its evidence excerpt, source URL and publication date.

Structured outputs that keep each claim tied to its source make merging dependable and preserve provenance. Shorter prose still loses structure. Follow-up rounds add latency to fix a design flaw. Re-reading every source duplicates work.

Question 11 · Answer: D

Resume the named session and tell Claude exactly which two files changed so that it re-analyses them specifically.

Most of yesterday's context is still valid, so resuming keeps it. Pointing Claude at the changed files avoids it reasoning from stale results. A fresh start throws away valid work. Claude doesn't automatically know which files changed since earlier tool results were captured. Forking is for exploring alternatives, not refreshing stale data.

Question 12 · Answer: C

Go ahead with synthesis using the four successful results, optionally try an alternative approach for the patent search, and state the patent coverage gap clearly in the report.

Graceful degradation keeps the valid work and is honest about what is missing. Restarting everything wastes successful results. Making up findings is fabrication. Waiting indefinitely may never finish.

Question 13 · Answer: A

A post-tool-use hook (or tool-side filtering) that redacts the sensitive fields from results before they are added to the conversation.

To keep data out of the context, you have to remove it before it gets there, deterministically. A prompt instruction still lets the data into the context. Customer behaviour can't control what a legacy system stores. Encryption at rest doesn't stop the model from seeing and processing the data.

Question 14 · Answer: A

Have the coordinator give each subagent an explicit, non-overlapping scope (for example by subtopic or source type) in its prompt.

Overlap starts with how the coordinator defines the subtasks, so that is where to fix it. De-duplicating later still pays for the duplicated work. Running them in sequence gives up parallelism. A single agent gives up the benefits of the multi-agent design.

Question 15 · Answer: D

Let the model choose tools based on the context of each case, and enforce the two invariants programmatically (for example with prerequisite gates and argument checks).

The strongest design combines the model's flexibility for varied cases with deterministic guardrails for things that must never be broken. Decision trees are brittle when cases vary. Invariants enforced only by prompt can fail. Leaving refund amounts unchecked breaks one of the invariants.

Question 16 · Answer: C

Detect repeated identical tool calls and return a tool result saying no new results are available and suggesting a different query or approach, with an overall iteration cap as a backstop.

Telling the agent in the tool result that it is stuck helps it change strategy, and the cap prevents runaway loops. Temperature is an indirect and unreliable lever. Returning fake results corrupts the research. Removing the tool after one call cripples legitimate searching.

Domain 2: Tool Design & MCP Integration

Question 17 · Answer: A

The system prompt's blanket instruction overrides the tool descriptions. Remove or narrow it so that it doesn't conflict with correct tool selection.

Instructions in the system prompt can steer tool selection against otherwise good descriptions. Look for conflicting directives such as "always" rules. Description length, name prefixes and array order aren't the issue here.

Question 18 · Answer: C

Redefine and rename the tools so their purposes don't overlap (for example, scope one to the product-specification store and one to the internal how-to wiki), and say in each description when to use it instead of the other.

Overlapping tool purposes cause confused selection. Clear, distinct purposes and when-to-use guidance fix the root cause. Always calling both doubles cost. Random tie-breaking is arbitrary. A third combined tool adds yet more overlap.

Question 19 · Answer: D

Return a structured business-rule error that is marked as not retryable and includes a customer-friendly reason. The agent explains the limit and offers escalation or other options.

Business-rule rejections aren't transient, so they shouldn't be retried. They need a clear reason the agent can pass on to the customer. Marking it transient wastes retries. A silent partial refund misleads the customer. A generic failure leaves the agent unable to explain or offer a next step.

Question 20 · Answer: A

Inside the subagent: retry locally with backoff, and pass the error up to the coordinator, with context and any partial results, only if it can't be recovered.

Transient errors are best handled close to where they happen, which keeps the coordinator's context free of noise. Errors that can't be recovered should then be passed up with enough context to decide what to do. Sending every retry to the coordinator adds noise. Asking the end user is premature. Returning stale cached data without saying so is misleading.

Question 21 · Answer: D

When you need team-specific workflows or internal systems that existing servers don't cover. Standard integrations like Jira should normally use the existing server.

Reusing a maintained server saves effort for standard integrations. Building custom pays off for workflows unique to your team. Neither "always custom" nor a blanket rule about community-server security is good practice; evaluate each server on its merits. Matching the number of API endpoints isn't a meaningful goal.

Question 22 · Answer: C

Tools from all three servers, discovered when Claude Code connects and available at the same time.

Scopes control where a server is configured and who shares it, and all configured servers contribute tools at the same time. Neither scope replaces the other wholesale, and you don't pick a single server per session.

Question 23 · Answer: B

Rewrite the MCP tool's description to explain what it can do (symbol-aware search, cross-references, what it returns) and when to prefer it over plain text search.

The model picks tools based on their descriptions. If the MCP tool's advantages aren't explained, a familiar built-in wins. Built-in tools can't be uninstalled simply, and removing them isn't the goal. Clashing names cause confusion. Depending on users to name the tool every time isn't scalable.

Question 24 · Answer: D

Glob with the pattern `**/*.tf`.

Glob finds files by path pattern, which is exactly what's needed. Grep searches file contents and would both miss .tf files that don't contain the word and match files that aren't .tf. Walking the tree by hand or using `cat` on everything wastes context.

Question 25 · Answer: C

Grep for entry points (such as the payment route handlers), then Read along the imports and calls step by step, building understanding as you go.

Targeted search followed by following the trace keeps the context focused on what matters. Reading everything uses up context on irrelevant code. Guessing from the README is unreliable. Alphabetical order doesn't follow the logic of the code.

Question 26 · Answer: B

`auto`*, which lets Claude decide whether to call a tool or reply with text.*

Conversational agents need to be able to reply in text, and `auto` allows that. Forcing tool calls is for pipeline steps that must produce structured output. A workaround tool adds needless complexity, rotating forced tools makes no sense, and parsing text instead of using tool calls goes backwards.

Question 27 · Answer: A

Return a validation error that states the expected format, with an example, so the agent can fix the input and retry.

Validation errors should be specific enough for the model to correct itself; an agent can easily resolve "next Tuesday" into a date. A generic error triggers unnecessary escalation. A silent default schedules the wrong date. Returning false success hides the failure.

Domain 3: Claude Code Configuration & Workflows

Question 28 · Answer: A

One rule with a glob such as `services/*/migrations/**` *applies to every matching file wherever it sits, so you don't need 12 duplicated directory-level files.*

Glob-scoped rules apply conventions that cut across directories from a single source of truth, and load only when matching files are involved. They aren't always loaded, they don't override everything else, and they aren't about files outside the repo.

Question 29 · Answer: A

In the developer's user-level `~/.claude/CLAUDE.md`.

Personal working preferences belong in user-level memory, which applies across that person's projects and doesn't affect anyone else. Project files are shared with the team. A rule scoped to everything would apply to everyone. Settings files control configuration, not behavioural instructions.

Question 30 · Answer: A

Move the checklist into a skill (for example `.claude/skills/release/SKILL.md`*) that is invoked when needed.*

Content that is always relevant belongs in CLAUDE.md. Procedures needed only now and then belong in skills, which load on demand. Cutting it down loses needed steps. User-level files still load every session and aren't shared. Compacting doesn't stop the file from being loaded.

Question 31 · Answer: A

An `allowed-tools` *entry in the SKILL.md frontmatter listing only read and search tools.*

`allowed-tools` limits which tools are available while the skill runs, which enforces read-only behaviour. Instructions in the body or in CLAUDE.md are advisory. A pre-commit hook acts too late, after files have already been changed in the working tree.

Question 32 · Answer: D

`argument-hint`*, which tells people the skill expects a ticket ID (for example* `<ticket-id>`*) when they invoke it.*

`argument-hint` shows users which parameter the skill expects. Forked context isolates the run but doesn't prompt for input. Tool restrictions limit capabilities, not inputs. A stronger model still can't reliably infer a ticket ID nobody gave it.

Question 33 · Answer: C

Create a personal skill under `~/.claude/skills/` *with a different name, such as* `review-strict`.

A personal skill with its own name gives you the variant without touching the team's skill or creating a confusing name clash. Editing the shared skill changes it for everyone. Reusing the same name invites confusion. Project CLAUDE.md is shared with the team.

Question 34 · Answer: C

Delegate the exploration to the Explore subagent so the verbose discovery stays in its own context and only a summary comes back.

Subagents isolate high-volume exploration and return condensed findings, keeping the main context free for planning. Skipping planning on a major refactor is risky. Pasting the whole codebase fills the context even faster. Letting the context run out loses work.

Question 35 · Answer: D

Ask Claude to interview you: have it ask questions that bring out considerations such as consistency requirements, TTLs and failure behaviour before it proposes a design.

When you're unfamiliar with a domain, having Claude question you surfaces requirements you might not have thought of, before they are baked into code. Implementing immediately risks building the wrong thing. Writing the spec alone misses Claude's knowledge of the domain. Merging random implementations doesn't resolve the open questions.

Question 36 · Answer: A

Write tests first that cover the expected behaviour and edge cases, then have Claude implement the function and iterate by feeding it the test failures.

Test-driven iteration gives Claude an objective target and concrete failure feedback to converge on. Self-assessment isn't verification. One typical input doesn't exercise edge cases. Tests written to match existing behaviour only lock in whatever bugs it has.

Question 37 · Answer: C

In a single detailed message that describes all three and how they interact, so that the fixes are designed together.

Interacting problems need to be dealt with together, or one fix may undo or conflict with another. Sending them one at a time suits independent issues. Fixing only one ignores known defects. Parallel tasks without shared context invite conflicting fixes.

Question 38 · Answer: A

Include the relevant existing test files in context, and document your testing standards and available fixtures in the project CLAUDE.md, which the CI run reads.

Claude can only avoid duplicates and use your fixtures if it can see them and knows your conventions; CLAUDE.md gives that project context to CI runs too. Running tests twice doesn't detect duplicates. More time doesn't guarantee it looks in the right places. Manual clean-up defeats the purpose of automation.

Question 39 · Answer: A

A post-tool-use hook in the project's `.claude/settings.json`*, matching the Edit and Write tools, that runs the formatter.*

Hooks run deterministically at lifecycle events, so they suit things that must always happen. A CLAUDE.md instruction is advisory and can be forgotten. A manual skill or command depends on people remembering to run it, and formatting the whole repository is wasteful.

Domain 4: Prompt Engineering & Structured Output

Question 40 · Answer: B

"Flag a comment only when what it says the code does contradicts what the code actually does." Add an example of a contradiction and an example of acceptable imprecision.

Precise criteria and examples define what counts as a real issue. "Important" is still vague. Reporting everything at low severity keeps the noise. Dropping the check misses comments that are actually misleading, which can cause real bugs.

Question 41 · Answer: B

Define explicit criteria for each severity level, with concrete code examples for each.

Inconsistent labels come from criteria that aren't defined. Definitions anchored in examples make labelling repeatable. Asking it to "think harder" doesn't create a standard. Removing severity loses prioritisation. Averaging three runs of an undefined standard is still undefined.

Question 42 · Answer: C

Few-shot examples of borderline cases that show the reasoning for choosing one action over a plausible alternative.

Few-shot examples that show the reasoning on ambiguous cases teach the model where the line is in a way it can generalise. A blanket default fails whenever escalation is actually right. Better descriptions help but are less effective for subtle boundaries. Higher temperature makes the results less consistent, not more.

Question 43 · Answer: D

Add few-shot examples that show how to extract each kind of quantity, including recording informal amounts as stated, with a flag, rather than converting them.

Examples that show how to handle the varied structures (including keeping imprecise amounts honestly) reduce both omissions and fabrications. Forced conversion invents precision that isn't there. Dropping recipes loses data. A required field pushes the model towards making up a value.

Question 44 · Answer: D

Replace them with a diverse set of examples covering different layouts and edge cases, so the model learns the general task rather than one template.

Few-shot examples should represent the variety of real inputs. Near-identical examples encourage the model to copy surface patterns. More of the same makes the overfitting worse. Removing the examples gives up their benefit. A separate classifier adds machinery without fixing the examples.

Question 45 · Answer: A

Add explicit normalisation rules to the prompt and schema (for example ISO 8601 dates and ISO 4217 currency codes, with format patterns or enums where possible), with examples.

A schema field typed as "string" doesn't specify a format. Explicit normalisation rules, enforced with patterns or enums where possible, make the output consistent. Leaving it to consumers spreads the work and the inconsistency. Separate fields per format make things worse. Temperature doesn't define a standard.

Question 46 · Answer: D

The original document, the extraction that failed, and the specific validation errors found.

Specific error feedback, together with the source and the previous attempt, lets the model correct the actual mistake. Sending the same prompt again just repeats the attempt. Without the document, the model can't re-check the facts. A generic "wrong" message gives it no direction.

Question 47 · Answer: B

Extract both `stated_total` *and* `calculated_total` *(the sum of the line items), plus a* `totals_conflict` *flag, so that mismatches are visible and can be routed for review.*

Capturing what the document says alongside what the numbers add up to shows conflicts instead of hiding them, and supports review. "Fixing" errors quietly changes the record. Computing totals only downstream loses the stated value you need for auditing. Dropping invoices loses data that matters.

Question 48 · Answer: D

A batch request can't run a multi-turn tool loop: you can't run a tool and return its result within the same batch request, so agent loops don't fit the batch model.

Batch requests are processed independently and asynchronously, so there is no chance to execute a tool and continue mid-request. Tool definitions are allowed, batches cost less rather than more, and processing can take up to 24 hours rather than finishing in under a minute.

Question 49 · Answer: D

Submit them through the Message Batches API for the roughly 50% cost saving, since the workload tolerates delay.

Large, non-urgent, independent jobs are what batch processing is for, and the discount is significant. Synchronous calls cost more for no benefit here. Hand sampling loses coverage and consistency.

Question 50 · Answer: D

Calibrate the confidence values against a labelled validation set, so that you know how the scores actually relate to correctness.

Confidence values reported by the model are not inherently calibrated. You need labelled data to find out which thresholds actually separate good findings from bad. Using the raw numbers or an arbitrary 0.5 assumes calibration that hasn't been measured. Asking for honesty doesn't calibrate anything.

Question 51 · Answer: A

Put the long document near the top, wrapped in clear XML-style tags, and put the instructions and question after it, at the end of the prompt.

With long inputs, putting the documents first and the query last, with structural tags separating content from instructions, tends to improve quality. Repeating instructions bloats the prompt. Splitting by page loses clauses that span pages. Burying the question in the middle hurts attention.

Domain 5: Context Management & Reliability

Question 52 · Answer: A

Record key findings in a scratchpad file as you go and refer to it, give large sub-investigations to subagents, and use `/compact` *deliberately.*

Long sessions degrade as context fills up. Writing down findings, isolating heavy exploration in subagents and compacting deliberately all preserve specifics. Hoping for recall ignores the problem. Restarting on a timer throws away context. Pasting the transcript back makes the context even larger.

Question 53 · Answer: B

Have each agent save its state (completed subtasks, findings, pending work) to a structured manifest. On restart, the coordinator loads the manifest and passes the relevant state into the agents' prompts.

Explicit, structured checkpoints make it possible to resume. Models don't remember anything between runs. Speed doesn't prevent crashes. Replaying raw logs is costly and doesn't give a clean resumable state.

Question 54 · Answer: B

Replace the confidence threshold with explicit escalation criteria based on case characteristics, and add few-shot examples of when to escalate and when to resolve.

Self-rated confidence doesn't track actual difficulty, as the audit shows. Criteria based on what each case involves are reliable. Moving the threshold leaves the miscalibration in place. Rating twice doesn't fix it either. Blanket rules by category ignore the variation within each category.

Question 55 · Answer: C

Acknowledge the frustration and offer the concrete fix straight away, and escalate if the customer then says they would prefer a human.

Frustration alone doesn't call for escalation when the agent can resolve the problem. Empathy plus a quick fix is usually best, while still honouring any request for a human. Sentiment-triggered escalation sends solvable cases to the human queue. Ignoring the tone or telling the customer to calm down makes things worse.

Question 56 · Answer: C

Escalate to a human, because this is a gap in policy that needs a human decision. Tell the customer it is being reviewed.

When policy doesn't cover a situation, the agent shouldn't make up policy in either direction. A gap in policy is a standard escalation trigger. Agreeing, refusing or inventing a compromise all amount to the agent making policy on its own.

Question 57 · Answer: B

Mark which findings are well supported and which rest on weak or single sources, and state which areas weren't covered because sources were unavailable.

Readers need to know how strong the evidence is and where the gaps are in order to use a report responsibly. Equal certainty misleads. Silently leaving things out hides gaps. A general disclaimer doesn't tell readers which claims to trust.

Question 58 · Answer: B

Require subagents to include publication or data-collection dates with every claim in their structured output, so that synthesis can see the figures differ because of time, not because one is wrong.

Temporal metadata turns an apparent contradiction into a meaningful trend. Picking the higher figure, quietly dropping older data or averaging across years all discard or distort information.

Question 59 · Answer: B

Route documents with low-confidence or ambiguous fields (using thresholds calibrated on labelled validation data) to review, plus a random sample of high-confidence ones to catch blind spots.

Confidence-based routing focuses scarce reviewer time where errors are most likely. Calibrated thresholds make it trustworthy, and sampling high-confidence documents catches systematic errors that the scores miss. Purely random review wastes effort. Reviewing only the lowest-scoring documents leaves confident errors unchecked. Choosing by customer age has nothing to do with error risk.

Question 60 · Answer: B

Have upstream agents return compact, structured results (key facts, sources and any flags) and leave out their full reasoning, so the downstream context stays focused.

When downstream context is limited, upstream agents should send only what is needed, in a structured form. A larger window postpones the problem and still dilutes attention. Truncating arbitrarily loses findings. Removing synthesis gives up integration and quality.

Score Sheet

Fill in after marking. The real exam reportedly uses a scaled score with 720/1000 to pass; 75%+ in every domain on this set is a good sign you're ready.

Domain	Questions	Correct	%	Topics to revisit
D1. Agentic Architecture & Orchestration	16			
D2. Tool Design & MCP Integration	11			
D3. Claude Code Configuration & Workflows	12			
D4. Prompt Engineering & Structured Output	12			
D5. Context Management & Reliability	9			
Total	**60**			